

The Linux Unicode way

In Windows, you can use *Alt+number* pad key combinations to type Unicode characters. In Linux, you need to first press *Ctrl+Shift+U*. The cursor will temporarily transform to an underlined *u*. Then, when you type the codepoint number and press the *Enter* key, the corresponding Unicode character will be inserted at the text cursor. To type the copyright symbol (©), you need to type *Ctrl+Shift+U*, *a9*, and then *Enter*. For the Indian rupee symbol (₹), you need to type *Ctrl+Shift+U*, *20B9*, and then *Enter*.

Linux offers an additional way of typing some characters using a ‘Compose’ key. This is a modifier key that is designated as such in the keyboard settings. On my computer, I have set the useless *Windows* key as its *Compose* key (*Preferences » Keyboard » Layouts » Options » Compose key position » Left Win*). Now, to type the copyright symbol, I type *Windows+O* (simultaneously) and then *c* (subsequently).

The trouble with space

In my first year of self-publishing (2020-21), I created 21 books. I wrote, edited, illustrated, designed and formatted them myself — all thanks to open source software. For every book, I built a PDF file for the paperback (using a shell script) and an EPUB file for the ebook (manually). I wrote the manuscript in the text-only CommonMark format. (This is a new standardised dialect of the old Markdown format.) I used the CommonMark executable to convert the text manuscript to HTML. Then, I styled the HTML with some CSS. This stylised HTML was fed to KhtmlToPDF to create the paperback PDF file. (I used Calibre to create the ebook copy from the same HTML.) However, the PDF output had a few problems.

The current KhtmlToPDF was resurrected by an Indian from an almost-abandoned open source project. KhtmlToPDF uses a headless Firefox browser of considerable vintage. It prints

Key	Description	First key press	Second key press
©	Copyright	 o	c
®	Registered	 o	r
™	Trademark	 t	m
€	Euro	 =	e
£	Sterling pound	 =	l
á	'a with accent' as in <i>á la</i>	 '	a
œ	'a with e' as in <i>amoeba</i>	 o	e
½	half	 1	2
⅙	one-sixth	 1	6
☺	smiley face	 :	)
☹	frowning face	 :	(

Figure 6: This table shows the Compose key as . You will find more comprehensive lists of Linux compose-key combinations online. I have included a big one in my book *Linux Command-Line Tips & Tricks*.

Codepoint	Description
2002	EN SPACE
2003	EM SPACE
2004	THREE-PER-EM SPACE
2005	FOUR-PER-EM SPACE
2006	SIX-PER-EM SPACE
2007	FIGURE SPACE
2008	PUNCTUATION SPACE
2009	THIN SPACE
200A	HAIR SPACE
...	...

Figure 7: These are a few of several types of space characters in Unicode

(not displays) the input HTML page. (KhtmlToPDF provides plenty of options for controlling the pagination and writing auto text such as page numbers, headers and footers, before exporting the print output to PDF using iText.) The printing talents of the Firefox browser are good but not perfect. Randomly, the space after an italicised word would disappear. If I manually added an extra space, it would be too much and the blank space would stand out. Initially, a fix seemed hopeless. But then, Unicode came to my rescue.

Unicode offers several types of space characters. There is space about the size of an ‘n’ — the en space, and also a space about the size of an ‘m’ — the em space, both of which may seem familiar if you know CSS. But, that is not all. I experimented with several other Unicode space characters and, eventually, I could hide KhtmlToPDF’s deficiencies.

In some places, the last letter in a line would be partially displayed. The fix for that was to use the non-breaking space character (*Compose + Space + Space*) after the word, and it would conveniently wrap to the next line. Sometimes, I did not want a line to break at a hyphen. Conveniently, Unicode has a non-breaking hyphen (*Ctrl+Shift+u* and *2011*) as a fix.

That’s not all

You can already see that I am now addicted to the long hyphen (—) but you may not have noticed that instead of three dots (...), I have used an ellipsis (...). This is just one Unicode character, instead of three. I do not use apostrophes for my quotations. There are dedicated Unicode characters for it. I type them with the *Compose* key. I type ‘Hi’ instead of ‘Hi’. I write “Hello, World”, instead of "Hello, World". But, not when I write code. When other writers write source code in Microsoft Word, the annoying software autocorrects all apostrophes and double quotation marks into Unicode quotation marks. Such a code listing will not compile in an IDE. It becomes a nightmare for the person who does the page setting. My manuscripts (including the code listings) are entirely in text (as CommonMark). What is written in text, stays in text. So whether it is exported to HTML, Word or PDF, the code will always compile. **END** 🐧

By: V. Subhash

The author has written 22 books including *FFmpeg Quick Hacks (free)*, *CommonMark Ready Reference*, *Linux Command-Line Tips & Tricks*, *Cool Electronic Projects* and *How To Install Solar*.